DES646: COURSE PROJECT REPORT

GESTURA: Because every gesture deserves to be understood

Team-AceOfSyndicate

1 Introduction

This course project revolves around the design and development of a solution that addresses a specific challenge through thoughtful, user-centered innovation. Emphasizing core design principles, the project involved systematic research, continuous iteration through prototyping, and rigorous testing to ensure that the final product is both highly functional and user-friendly. GESTURA is aimed at making the lives of individuals whose sentences often go unheard or unrecognized much easier by providing an accessible and efficient means of communication. The goal was to create a tool that not only detects and interprets sign language gestures accurately but also offers an intuitive interface for seamless interaction.

I chose to work on a project like GESTURA because I have always been deeply interested in exploring the field of Computer Vision within my broader machine learning journey. This project provided a perfect opportunity and an added incentive to delve into practical applications of computer vision technology. Additionally, I have often encountered, heard, and read about the challenges faced by differently-abled individuals, particularly in the context of communication. Many face difficulties in expressing themselves effectively in everyday interactions, which can lead to feelings of frustration and social exclusion. By developing GESTURA, I hope to contribute in even a small way towards alleviating these challenges, at least for some members of the ASL-speaking community. My aim is not only to apply my technical skills but also to create a meaningful tool that fosters greater inclusivity and accessibility, ultimately helping individuals to communicate more confidently and comfortably in a variety of settings.

2 Research

The Gestura project addresses the challenge of facilitating communication for individuals who use American Sign Language (ASL) by developing a virtual keyboard that translates ASL signs into printed letters. This user-centered design leverages advances in computer vision, specifically landmark based hand tracking technology such as MediaPipe, to detect and classify isolated ASL alphabet signs in real time. Sign language recognition research has witnessed significant progress with the integration of deep learning models, including convolutional neural networks (CNNs) and recurrent architectures, improving accuracy for both static and dynamic gestures. However, accurately capturing subtle hand and finger positions remains challenging due to variability in lighting, hand orientation, and signer differences. Gestura follows a landmark driven approach to balance model complexity with real-time performance, drawing upon datasets annotated with hand keypoints to support robust recognition under diverse conditions. This approach is widely supported in recent literature as an effective method for building accessible interaction tools, such as gesture-based virtual keyboards, that serve differently-abled users.

Despite encouraging results in constrained environments, current methods, including those used in Gestura face limitations in practical, real world settings where occlusion, background clutter, and signer diversity impact accuracy. While many existing systems achieve high performance on standard ASL alphabets, their generalization outside controlled conditions remains an open challenge. The project design incorporates user-centered considerations such as low latency, simplicity of the interface, and clear feedback, recognizing that usability is as vital as accuracy for acceptance by the ASL community. Ethical aspects, including privacy in image capture and inclusivity in testing with diverse users, guide the design process. Future improvements may expand capabilities to continuous sign sequences and multimodal cues, leveraging transformer-based temporal models and more extensive, diverse datasets. Gestura thus aligns with ongoing research trends by adopting a pragmatic, scalable approach that aims to make meaningful contributions to accessible communication technology while serving as a foundation for future innovation.

3 Modules and Packages

Here is a list of all the pre-existing modules and packages that I used in this project. All of the code was written locally on my ersonal device using Visual Studio Code by using a virtual environment-'ugh', where all of these dependencies can be found installed.

Additionally, I also used a pre-existing model from **tensorflow.keras** for prediction and a module called **cvzone** for detecting hands and marking landmarks. I also tried training a model on Teachable Machines. However, these files were deleted beforehand because the inferences were not satisfactorily made. For help with the code, **Perplexity.ai** was used.

Numpy	Tqdm	CustomTkinter
Scikit-learn	Random	autocorrect
Opencv-python	PIL	PySimpleGUI
Mediapipe	Tkinter	
Pickle	Threading	

4 Dataset(s) and Handling

For this project, two datasets were used for diversity. One of which is the standard one, directly downloaded from Kaggle and the other is a custom dataset, recorded using my own hands.

4.1 Dataset-1 ASL Alphabet from Kaggle

The primary dataset used for the Gestura project was the ASL Alphabet dataset from Kaggle, which contains tens of thousands of labeled images representing each letter of the ASL alphabet (A-Z), as well as additional classes for "space," "delete," and "nothing". Each class folder includes numerous images of hands performing the respective sign, offering valuable variety in gesture pose and lighting. For this project, only images with visible and clearly defined hand signs were used, filtered via MediaPipe's hand detector to maximize training quality and minimize noise from irrelevant or blank images.

While this dataset provided a strong foundation for model development and fast prototyping, it also introduced certain challenges. Differences in background, lighting, and hand orientation made it hard for the model to recognize some letters consistently. Additionally, the dataset's quality and lack of signer diversity sometimes limited generalization to real-world conditions. Illustrative examples, such as those for the letters I, L, and Y, display the range of hand shapes and postures needed for accurate recognition. These experiences highlighted the importance of high-quality, varied data and suggest the benefit of future work on dataset augmentation and broader data collection to increase system robustness and accuracy.

Letter I

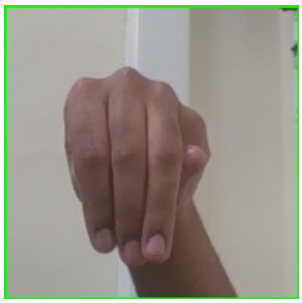
Letter L

Letter Y

4.2 Dataset-2 Custom-made dataset

After observing lower accuracy with models trained on the Kaggle ASL dataset, a custom dataset was created to ensure clearer, more controlled images tailored to the requirements of the Gestura project. Using a script built with the OpenCV-python library, a fixed 200x200 region of the webcam feed was captured while different ASL signs were performed within that region. Each new frame was saved by pressing a key, resulting in approximately 850 photos for each hand sign. This targeted approach helped standardize image quality, background, and hand orientation, directly addressing several inconsistencies seen in the public dataset.

All captured images were systematically organized by letter and indexed for ease of training and model evaluation. Representative samples—for example, the signs for letters M, U, and D—demonstrate the clarity and consistency of the new data. This custom dataset allowed for more robust hand recognition and provided valuable data for iteratively refining the model. By directly addressing the shortcomings of uncontrolled, varied public datasets, the custom collection significantly improved both the training workflow and the accuracy of ASL gesture detection for the project.



Letter M

Letter U

Letter D

4.3 Handling/Preprocessing

The images collected from both the Kaggle and custom-made datasets were systematically organized as nested folders, each containing labeled examples for every sign. To streamline preprocessing, MediaPipe was first used to automatically detect and isolate hands in each image, saving cropped versions for further analysis. Hand landmarks gave detailed coordinate points provided by MediaPipe were then extracted and stored alongside their corresponding images in a .pickle file. This process not only accelerated downstream data loading but also ensured consistency when feeding data into the gesture recognition model.

One notable challenge in this workflow was the variance in the number of images per class, which stemmed from the inherent complexity of some ASL signs and differences in webcam quality. To minimize the risk of bias during

training, a balanced subset of images was selected for each letter class, resulting in a total of 1400 images from the first dataset and 610 from the second. Organizing these records under a new .pickle file further improved reproducibility and model reliability. By harmonizing image counts and standardizing landmark extraction, the preprocessing pipeline enabled fair and effective training, setting a solid foundation for robust ASL detection.

5 Model training

Initially, the project adopted a deep learning approach using tensorflow.keras models for gesture classification. Hand detection and landmark extraction were first performed using the cvzone module. For model training, the Teachable Machines platform was utilized, where images of each sign were uploaded and various hyperparameters (such as batch size and epochs) could be adjusted. This platform facilitated accessible model training and live inference demos, but models exported from Teachable Machines in .keras format proved incompatible with some recent TensorFlow and Python environments. Moreover, empirical testing revealed that these deep learning models, despite their sophistication, did not yield satisfactory results for all classes within the available datasets.

To overcome these obstacles, the project shifted towards using classical machine learning algorithms, prioritizing simplicity and interpretability. The RandomForestClassifier from the sklearn library was selected due to its robustness and ease of implementation. Datasets were divided into training and testing splits (80%/20%), and model evaluation was carried out using the accuracy-score metric from sklearn. Results were highly promising, with the classifier achieving an accuracy of approximately 99% on both the Kaggle and custom-collected datasets. The final trained models were then saved for real-time inference, confirming that even traditional algorithms, when paired with effective preprocessing and high-quality landmark features, can deliver impressive results for ASL letter recognition tasks.

6 Real-time Inference

To rigorously evaluate the model's practical capabilities, a real-time inference program was developed. This application activates the webcam, detects a single hand in the frame, and continuously extracts hand landmarks using MediaPipe, storing them in a rapidly updated NumPy array. Each new array snapshot is fed into the trained classifier, instantly predicting the detected sign and displaying both the letter and its associated confidence score on screen. This interactive interface allowed for straightforward, hands-on testing and rapid assessment of the model's performance in real-world conditions, simulating the intended use-case for end users. Here is a picture of the interface:

Models trained on the custom dataset (Dataset-2) consistently outperformed those trained on the original Kaggle dataset (Dataset-1) in these real-time eval-

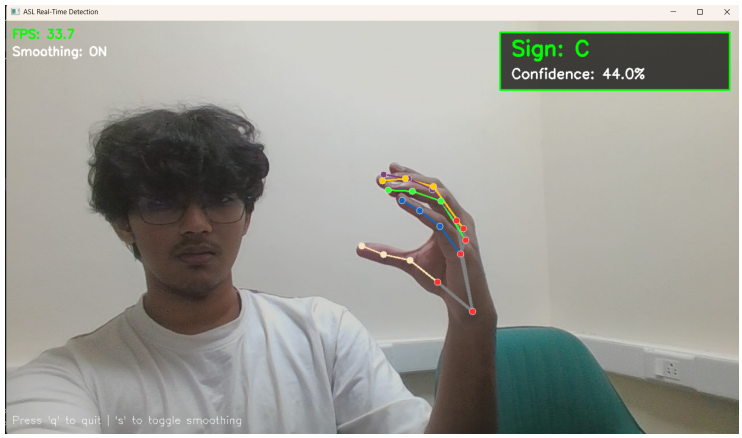


Fig: Inference Interface

uations. By sequentially presenting all 28 possible ASL signs multiple times to both models, it was evident that the Dataset-2 model achieved higher accuracy and more reliable predictions. The controlled environment, consistent backgrounds, and tailored image quality of the custom dataset clearly contributed to improved recognition, reducing misclassification and boosting user trust in the interface's feedback during live use.

As mentioned in the log, these are the most significant discrepancies

- The letters A and S were overshadowed by the letter E because of the similarity in the signs.
- The letter O was quite rarely detected and was overshadowed by C as they are almost the same signs.
- Sometimes, the letters M, N and T were not being detected as the hand was not detected by mediapipe.
- The letter Q was inaccurately detected, probably because of camera quality or sign complexity.

7 Making of the keyboard application

7.1 Simple Python Program

The implementation phase began by leveraging the trained model to build a virtual keyboard capable of translating ASL hand-signs into English text. The application captures a frame from the webcam every two seconds and checks for the presence of a hand. If a hand is detected, its landmarks are extracted while the user performs a sign, and these landmarks are stored in a NumPy array. The array is then fed into the trained classifier, which predicts the hand-sign label;

this label is subsequently appended to a running list representing typed output. Special labels for “space” and “delete” allow users to construct words and edit text, mimicking a standard keyboard’s functionality. This system streamlines ASL-to-English translation in real time, empowering users to input sentences seamlessly using only their hand signs.

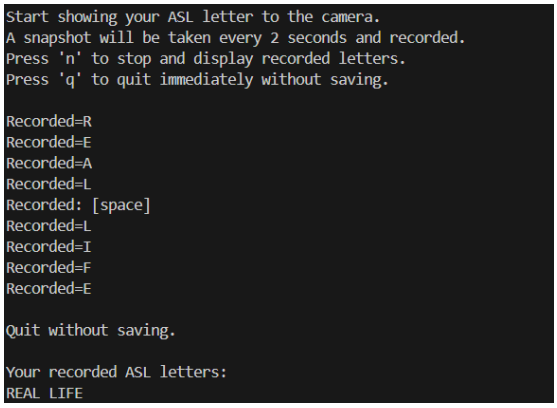


Fig: Keyboard-Python

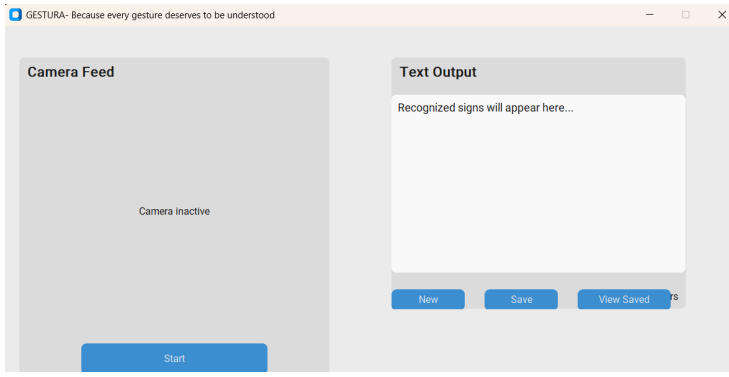
7.2 User Interface

To further enhance the program and bridge the gap between functionality and usability, a significant effort was made to refine the user interface. While PySimpleGUI initially seemed promising for its rapid prototyping capabilities, its licensing model ultimately proved restrictive. This prompted a shift to Tkinter, Python’s default GUI toolkit, which allowed for the rapid development of a working demo but fell short of providing a modern or visually appealing experience. Eventually, a transition was made to customtkinter, a more advanced package featuring contemporary UI elements like stylized buttons and layouts that more closely resemble actual keyboard applications. This upgrade brought a noticeable improvement in user experience and interface aesthetics.

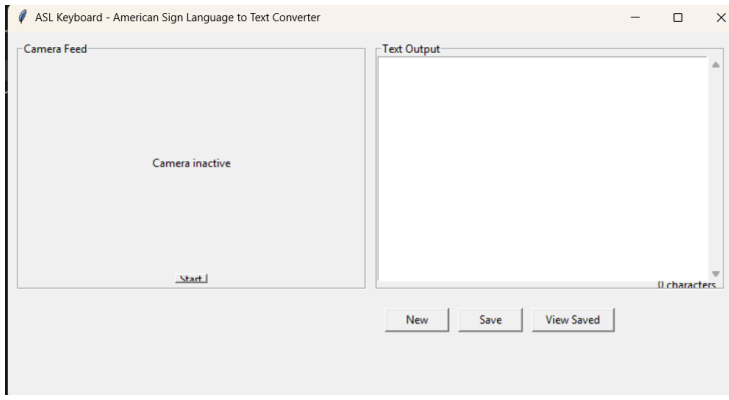
Despite the UI improvements, practical trials revealed several limitations. The customtkinter-powered keyboard application struggled to match the seamless response of the core keyboard-python program, often due to constrained camera resolution, which negatively impacted landmark accuracy. These technical hurdles made the input less reliable during live use. To further refine the experience, a spell-checking module from Python’s autocorrect library (Speller) was integrated, automatically correcting errors in the recognized letter sequences. Additionally, a save feature was implemented to allow users to archive their typed output for future reference, enhancing both usability and practical value.

Looking ahead, achieving even better performance and user satisfaction will require balancing technical constraints with thoughtful interface design. Improvements such as higher resolution image capture, additional error correction

mechanisms, and ongoing refinement to modern UI libraries like customtkinter could yield a smoother and more accessible tool. Continued emphasis on user feedback, accessibility, and integration of advanced features will help transform the virtual keyboard into a robust assistive technology for ASL users.



(a) Fig. Custom Tkinter



(b) Fig. Tkinter

Figure 1: Python GUI

8 Reflection

One of the most rewarding aspects of working on the Gestura project was the meaningful purpose behind it. Although gesture recognition and ASL-to-text translation have been explored by many researchers and multiple patents exist in this domain, I chose to pursue this project driven by my strong interest in computer vision and machine learning. This project offered me the perfect opportunity to dive deep into these technologies while contributing to an important cause—enhancing communication accessibility for the Deaf and hard-of-hearing communities. I am grateful to my instructor for providing motivation

and guidance throughout this learning journey.

In terms of strengths, the project successfully accomplishes its primary goal of detecting a majority of ASL signs and converting them into corresponding letters via a virtual keyboard interface. The system is functional and serves as a proof-of-concept for real-time ASL recognition paired with text output. However, the project is not without its challenges. Certain letters are more difficult to detect accurately, often requiring very specific hand angles or positions, which limits the model's robustness. Furthermore, the user interface design, while functional, remains quite basic and could benefit from more sophisticated integration to enhance user experience.

Most of the intended objectives and milestones were met, demonstrating the feasibility of the concept. Nevertheless, my limited expertise in advanced computer vision techniques and model optimization likely contributed to some discrepancies in accuracy and performance. Given more time and knowledge, I would have liked to invest further in refining the model, expanding the dataset, and improving the interface design to produce a more polished and reliable tool. Additionally, considering the project's topic is widely researched and not entirely original, I have decided not to submit an intellectual property disclosure or research paper. I sincerely appreciate the opportunity and support from my instructor and apologize for any shortcomings in originality.

This project has been an invaluable learning experience, expanding my technical skills, deepening my understanding of ASL, and highlighting the ongoing challenges and opportunities in accessible technology design. It has strengthened my enthusiasm for continuing work in machine learning applications that have practical and social impact.

9 Evaluation

The evaluation of Gestura reveals that while the project meets many of its intended goals, there are several areas where improvement is needed to enhance overall performance and user experience. A key strength of the project lies in its ability to detect a significant portion of ASL signs accurately enough to function as a virtual keyboard, demonstrating proof-of-concept feasibility. The integration of computer vision models with a real-time interface shows promise for practical use, especially in applications aimed at improving communication accessibility.

However, limitations in model accuracy and robustness were evident. The system struggled with consistent detection of certain letters, particularly those requiring precise hand orientations or subtle variations, which highlights the need for a more comprehensive and diverse training dataset. Lighting conditions and camera angles also impacted the model's performance, indicating that further work on preprocessing and environmental adaptability is essential. The user interface, while operational, lacked advanced design features that could enhance usability, such as visual feedback, error correction, or customization options.

The project's originality is another consideration. Gestura builds on well-established foundations in ASL recognition technology, so it does not represent a groundbreaking innovation; rather, it is a learning-focused implementation. This context shaped decisions around intellectual property disclosure and publication. Moreover, my current skill set placed limits on the depth of model tuning, dataset augmentation, and UI refinement possible within the available timeframe.

In future iterations, collaboration with experts in machine learning, ASL linguistics, and human-computer interaction could elevate the system's accuracy and user-friendliness. Additional time dedicated to data collection, model optimization, and user testing would likely lead to significant performance gains. Despite these challenges, Gestura has successfully served its educational purpose and lays a solid foundation for ongoing improvement and exploration in accessible communication technology

10 References and Links

American Sign Language Alphabets
Scikit-Learn
OpenCV
Mediapipe
Ref 1
Ref 2
Ref 3